

**UNIVERSIDAD NACIONAL AGRARIA DE LA
SELVA**

**ESCUELA PROFESIONAL DE INGENIERIA EN
CIBERSEGURIDAD**



ESTRUCTURA DE DATOS Y ALGORITMO

MODELOS MULTITHILOS EN C++

ESTUDIANTE:

ANA MARIA LEON ROSALES

DOCENTE:

CARLOS ABRAHAM RIOS RIVERA

CICLO: III

**1 de mayo – Tingo María
2026**

INTRODUCCIÓN

El uso de modelos multihilos es una técnica fundamental en el desarrollo de software moderno, ya que permite mejorar el rendimiento y la eficiencia de los programas mediante la ejecución concurrente de múltiples tareas dentro de un mismo proceso. Estos modelos permiten aprovechar mejor los recursos del sistema, especialmente en aplicaciones que requieren realizar varias operaciones de forma simultánea.

Los modelos multihilos definen la forma en que los hilos de ejecución son gestionados y planificados, tanto a nivel del programa como del sistema operativo. Entre los principales enfoques se encuentran la ejecución secuencial y la ejecución concurrente mediante hilos, los cuales constituyen la base para el diseño de aplicaciones más flexibles y escalables.

El presente informe tiene como objetivo analizar los modelos multihilos y presentar ejercicios prácticos en código que demuestren su funcionamiento. A través de estos ejemplos, se busca comprender cómo los hilos influyen en la ejecución de los programas y cómo pueden ser utilizados correctamente en el desarrollo de aplicaciones.

OBJETIVOS

Objetivo general

Analizar los modelos multihilos mediante la implementación de ejercicios en código, con el fin de comprender su funcionamiento y su impacto en la ejecución de los programas.

Objetivos específicos

- Identificar las características del modelo de ejecución secuencial.
- Explicar el funcionamiento del modelo multihilo concurrente.
- Analizar el modelo multihilo paralelo y sus ventajas.
- Implementar ejercicios prácticos en código que representen cada modelo.

DESARROLLO

MODELO SECUENCIAL (un solo hilo)

Definición

El modelo secuencial es el modelo de ejecución más básico y tradicional en la programación. En este modelo, las instrucciones de un programa se ejecutan de forma ordenada, una después de la otra, siguiendo el flujo definido por el programador. No existe paralelismo ni concurrencia, ya que todo el código se ejecuta en un solo hilo de ejecución.

Este modelo resulta sencillo de comprender, implementar y depurar, ya que el comportamiento del programa es predecible. Sin embargo, su principal desventaja es que no aprovecha los recursos modernos del hardware, como los procesadores multinúcleo.

Ejercicio

Contar cuántos números pares existen dentro de un arreglo de enteros.

```
ModeloSecuencial.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      // Declaración del arreglo de números
6      int numeros[] = {4, 7, 10, 3, 8};
7
8      // Variable para contar los números pares
9      int contadorPares = 0;
10
11     // Recorrido secuencial del arreglo
12     for (int i = 0; i < 5; i++) {
13         // Verifica si el número es divisible entre 2
14         if (numeros[i] % 2 == 0) {
15             contadorPares++; // Incrementa el contador
16         }
17     }
18
19     // Muestra el resultado final
20     cout << "Cantidad de numeros pares: " << contadorPares << endl;
21     return 0;
22 }
```

MODELO MULTITHILO CONCURRENTES

Definición

El modelo concurrente permite que varias tareas se ejecuten aparentemente al mismo tiempo mediante el uso de múltiples hilos. A diferencia del paralelismo, la concurrencia no garantiza que las tareas se ejecuten exactamente al mismo instante, sino que pueden alternarse en el tiempo.

Este modelo es especialmente útil para aplicaciones que requieren realizar múltiples tareas independientes, como servidores, sistemas interactivos o programas que manejan entrada y salida de datos.

Ejercicio

Ejecución de dos procesos concurrentes que imprimen mensajes.

```
ModeloConcurrente.cpp > ...
1  #include <iostream>
2  #include <thread>
3  using namespace std;
4
5  // Función ejecutada por el primer hilo
6  void procesoUno() {
7      for (int i = 0; i < 3; i++) {
8          cout << "Proceso uno ejecutandose" << endl;
9      }
10 }
11
12 // Función ejecutada por el segundo hilo
13 void procesoDos() {
14     for (int i = 0; i < 3; i++) {
15         cout << "Proceso dos ejecutandose" << endl;
16     }
17 }
18
19 int main() {
20     // Creación de dos hilos concurrentes
21     thread hilo1(procesoUno);
22     thread hilo2(procesoDos);
23
24     // Esperar a que los hilos terminen
25     hilo1.join();
26     hilo2.join();
27
28     return 0;
29 }
```

MODELO MULTITHREAD PARALELO

Definición

El modelo paralelo busca ejecutar tareas de forma simultánea aprovechando múltiples núcleos del procesador. Para ello, un problema grande se divide en partes más pequeñas que se ejecutan al mismo tiempo.

Este modelo mejora significativamente el rendimiento en tareas pesadas como cálculos matemáticos, procesamiento de datos y análisis de información.

Ejercicio

Sumar un arreglo dividiéndolo en dos partes.

```
ModeloParalelo.cpp > ...
1  #include <iostream>
2  #include <thread>
3  using namespace std;
4
5  // Función que suma una parte del arreglo
6  void sumaParcial(int arr[], int inicio, int fin, int &resultado) {
7      resultado = 0;
8      for (int i = inicio; i < fin; i++) {
9          resultado += arr[i];
10     }
11 }
12
13 int main() {
14     int datos[] = {1, 2, 3, 4, 5, 6};
15     int suma1, suma2;
16
17     // Ejecución en paralelo
18     thread t1(sumaParcial, datos, 0, 3, ref(suma1));
19     thread t2(sumaParcial, datos, 3, 6, ref(suma2));
20
21     t1.join();
22     t2.join();
23
24     cout << "Suma total: " << suma1 + suma2 << endl;
25     return 0;
26 }
```

MODELO MULTITHILO PRODUCTOR – CONSUMIDOR

Definición

El modelo productor–consumidor es un modelo de ejecución concurrente en el cual uno o más hilos, llamados productores, se encargan de generar datos, mientras que uno o más hilos consumidores se encargan de procesarlos. Ambos tipos de hilos comparten una estructura de datos común, como una cola o buffer. Debido a que los datos son compartidos, es necesario utilizar mecanismos de sincronización, como mutex, para evitar errores durante el acceso concurrente. Este modelo es ampliamente utilizado en sistemas donde la producción y el consumo de información ocurren de forma independiente.

Ejercicio

Un hilo genera datos y otro los consume utilizando una estructura compartida protegida para evitar conflictos.

```
ModeloProductor-Consumidor.cpp > ...
1  #include <iostream>
2  #include <thread>
3  #include <queue>
4  #include <mutex>
5  using namespace std;
6
7  queue<int> buffer; // Buffer compartido
8  mutex mtx;       // Mutex para sincronización
9
10 void productor() {
11     for (int i = 1; i <= 3; i++) {
12         mtx.lock();
13         buffer.push(i);
14         cout << "Produciendo: " << i << endl;
15         mtx.unlock();
16     }
17 }
18
19 void consumidor() {
20     for (int i = 1; i <= 3; i++) {
21         mtx.lock();
22         if (!buffer.empty()) {
23             cout << "Consumiendo: " << buffer.front() << endl;
24             buffer.pop();
25         }
26         mtx.unlock();
27     }
28 }
29
30 int main() {
31     thread p(productor);
32     thread c(consumidor);
33
34     p.join();
35     c.join();
36     return 0;
37 }
```

MODELO MULTITHILO MAESTRO – TRABAJADOR

Definición

Este modelo consiste en un proceso principal, llamado maestro, que se encarga de dividir el trabajo en varias tareas y asignarlas a distintos trabajadores. Cada trabajador ejecuta la tarea asignada y produce un resultado.

Este modelo permite organizar el trabajo de forma eficiente y aprovechar mejor los recursos del sistema, ya que varias tareas pueden ejecutarse al mismo tiempo. Es comúnmente utilizado en aplicaciones paralelas y sistemas que requieren procesar múltiples tareas similares.

Ejercicio

Asignar tareas a hilos trabajadores que se encargan de ejecutarlas.

```
ModeloMaestro-Trabajador.cpp > ...
1  #include <iostream>
2  #include <thread>
3  using namespace std;
4
5  // Función del trabajador
6  void trabajador(int numero) {
7      cout << "Cuadrado de " << numero << ": " << numero * numero << endl;
8  }
9
10 int main() {
11     int tareas[] = {3, 4, 5};
12
13     for (int i = 0; i < 3; i++) {
14         thread t(trabajador, tareas[i]);
15         t.join();
16     }
17     return 0;
18 }
```

MODELO DE HILOS INDEPENDIENTES

Definición

Este modelo se basa en la ejecución de varios hilos que funcionan de manera autónoma, sin compartir información ni recursos entre ellos. Cada hilo realiza su propia tarea sin necesidad de sincronización.

Este modelo es sencillo de implementar y reduce el riesgo de errores de concurrencia. Sin embargo, solo es adecuado cuando las tareas no necesitan comunicarse entre sí. Es común en aplicaciones donde se ejecutan procesos aislados o tareas simples en paralelo.

Ejercicio

Cada hilo se ejecuta de manera autónoma sin compartir información con otros hilos.

```
Modelo_de_Hilos_Independientes.cpp > ...
1  #include <iostream>
2  #include <thread>
3  using namespace std;
4
5  void procesoA() {
6      cout << "Proceso A activo" << endl;
7  }
8
9  void procesoB() {
10     cout << "Proceso B activo" << endl;
11 }
12
13 int main() {
14     thread h1(procesoA);
15     thread h2(procesoB);
16
17     h1.join();
18     h2.join();
19     return 0;
20 }
```


RESULTADOS

A partir del desarrollo de los ejercicios propuestos, se obtuvieron resultados que permiten comprender el comportamiento de los modelos multihilos en distintos escenarios de ejecución. En el modelo de ejecución secuencial, se observó que las instrucciones se ejecutan de manera ordenada y en un único hilo, lo que simplifica el control del flujo del programa, pero limita su rendimiento cuando se realizan múltiples tareas.

En el modelo multihilo concurrente, se evidenció que el uso de varios hilos permite ejecutar tareas de forma alternada, mejorando la capacidad de respuesta del programa y optimizando el uso del tiempo del procesador. Sin embargo, se identificó la necesidad de una correcta gestión de los hilos para evitar problemas como condiciones de carrera o inconsistencias en los datos.

Finalmente, en el modelo multihilo paralelo, los resultados demostraron un mejor aprovechamiento de los recursos del sistema, ya que los hilos pueden ejecutarse simultáneamente en diferentes núcleos del procesador. Esto se traduce en una mayor eficiencia y reducción del tiempo total de ejecución del programa.

CONCLUSIONES

Los modelos multihilos constituyen una herramienta fundamental en el desarrollo de software moderno, ya que permiten mejorar el rendimiento y la eficiencia de las aplicaciones mediante la ejecución concurrente de tareas. A través de los ejercicios desarrollados, se logró comprender cómo varía el comportamiento de un programa según el modelo de ejecución implementado.

El modelo de ejecución secuencial resulta adecuado para aplicaciones simples, mientras que los modelos multihilo concurrente y paralelo ofrecen ventajas significativas en escenarios donde se requiere realizar múltiples operaciones de manera simultánea. No obstante, el uso de hilos implica una mayor complejidad en el diseño del programa, lo que exige una correcta sincronización y administración de los recursos compartidos.

En conclusión, la correcta aplicación de los modelos multihilos permite desarrollar programas más eficientes y escalables, siempre que se comprendan sus características y se utilicen de manera adecuada según las necesidades del sistema.

REFERENCIAS

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.

Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson Education.

Herlihy, M., & Shavit, N. (2012). *The Art of Multiprocessor Programming*. Morgan Kaufmann.

Repositorio GitHub con los códigos desarrollados.

Disponible en:

https://github.com/AnitaLeon2025/Estructura_de_Datos_Algoritmo_2026